



Guia de operacion, respaldo y recuperacion

Sistema: Tiendita Maday

Cliente: La Familia

Version: 1.0

Fecha: 14 de julio de 2026

Audiencia: Propietario del servicio y responsable tecnico

Esta guia contiene valores recomendados. Los tiempos de recuperacion y retencion se vuelven compromisos solo cuando se escriben y firman en el acta o contrato de soporte.

1. Arquitectura operativa

Tiendita Maday se entrega como una aplicacion web de tres componentes:

Componente	Tecnologia	Responsabilidad
Frontend	Angular 21.2, aplicacion estatica/PWA	Interfaz para clientes y personal.
Backend	Node.js y Express 4	API, permisos y reglas de negocio.
Base de datos	PostgreSQL	Datos, transacciones, auditoria y migraciones.

El despliegue previsto utiliza un proyecto Railway con servicios separados para frontend, backend y PostgreSQL. El repositorio conserva Docker Compose para desarrollo local.

2. Responsabilidades

Actividad	Propietario del negocio	Responsable tecnico
Aprobar precios, roles y configuracion	Responsable	Apoya
Custodiar accesos de servicios	Responsable	Administra por delegacion
Despliegues y migraciones	Informado	Responsable
Respaldos y prueba de restauracion	Verifica	Ejecuta
Atender alertas operativas	Responsable	Apoya en fallas tecnicas
Renovar dominio y servicios	Responsable	Recuerda y verifica
Gestionar incidentes de seguridad	Decide	Contiene, investiga y documenta

3. Variables de configuracion

No copie secretos a Git, documentos, tickets publicos ni mensajes sin cifrar. Configure las variables directamente en Railway.

Variable	Servicio	Sensible	Uso
DATABASE_URL	Backend	Si	Conexion PostgreSQL.
JWT_SECRET	Backend	Si	Firma de tokens de acceso.
JWT_REFRESH_SECRET	Backend	Si	Firma de renovacion de sesion.
ACCESS_TOKEN_EXPIRATION	Backend	No	Vigencia; por defecto 2 horas.
FRONTEND_URL	Backend	No	CORS y enlaces de correo.
BACKEND_URL	Backend	No	URL publicada en OpenAPI.
PAYPAL_CLIENT_ID / PAYPAL_SECRET	Backend	Si	Cobro con PayPal.
PAYPAL_API_BASE	Backend	No	Sandbox o produccion.
PAYPAL_CURRENCY	Backend	No	Moneda; por defecto MXN.

Variable	Servicio	Sensible	Uso
GOOGLE_CLIENT_ID	Ambos	Parcial	Inicio de sesión con Google.
CLOUDINARY_URL	Backend	Si	Imágenes de productos.
EMAILJS_*	Backend	Si	Verificación y recuperación por correo.
API_BASE_URL	Frontend al compilar	No	URL pública de la API con /api.

El código contiene valores de desarrollo para JWT. En producción es obligatorio definir secretos largos y aleatorios; nunca se debe operar con los valores predeterminados.

4. Despliegue inicial en Railway

4.1 Base de datos

1. Cree un proyecto Railway y agregue PostgreSQL.
2. Inicialice una instalación nueva con `Arquitectura/Database/init.sql` usando la consola de datos o `psql`.
3. Confirme que el usuario de aplicación puede conectarse.
4. Registre el propietario de la cuenta y active controles de acceso disponibles.

4.2 Backend

1. Cree un servicio desde el repositorio.
2. Configure Root Directory como `mi cliente (la familia)/Arquitectura/Backend`.
3. Agregue las variables de la sección anterior.
4. Genere el dominio público.
5. Despliegue y compruebe `/y/api-docs`.

El comando de inicio ejecuta `npm run migrate` antes de levantar la API. El script aplica, en orden y una sola vez, los archivos SQL no registrados en `schema_migrations`.

4.3 Frontend

1. Cree otro servicio desde el mismo repositorio.
2. Configure Root Directory como `mi cliente (la familia)/Arquitectura/FrontEnd`.
3. Defina `API_BASE_URL` con la URL del backend y el sufijo `/api`.
4. Defina el cliente de Google si se utiliza esa autenticación.
5. Genere el dominio y compile la configuración de producción.
6. Regrese al backend y limite `FRONTEND_URL` al dominio final.

4.4 Comprobación posterior

- [] El backend responde con el mensaje de API activa.
- [] `/api-docs` abre y utiliza el servidor correcto.
- [] El frontend carga desde una ventana privada.

- [] Un administrador puede iniciar sesion.
- [] Una consulta del catalogo funciona.
- [] Los origenes no autorizados son rechazados por CORS.
- [] El correo de verificacion/recuperacion llega.
- [] PayPal esta en el modo esperado: sandbox o live.
- [] Cloudinary guarda y entrega una imagen de prueba.

5. Actualizaciones y migraciones

1. Revise cambios, notas de version y respaldo reciente.
2. Ejecute pruebas y compilacion en la revision que se desplegara.
3. Registre el hash del commit.
4. Despliegue primero el backend si incluye migraciones compatibles.
5. Verifique en logs que las migraciones terminaron sin error.
6. Despliegue el frontend con API_BASE_URL correcto.
7. Ejecute la lista de comprobacion posterior.
8. Conserve la revision anterior disponible para rollback.

No interrumpa una migracion. Si falla, capture el error completo y no intente marcar manualmente el archivo como aplicado.

6. Politica recomendada de respaldo

Elemento	Recomendacion inicial
RPO, perdida maxima de datos	24 horas, reducir si el volumen de ventas lo exige.
RTO, tiempo objetivo de recuperacion	4 horas durante horario de soporte.
Respaldo logico PostgreSQL	Diario.
Retencion	7 diarios, 4 semanales y 6 mensuales.
Cifrado	En transito y en reposo.
Ubicaciones	Proveedor y una copia fuera de Railway.
Prueba de restauracion	Trimestral y antes de cambios de alto riesgo.
Responsable	Nombre explicito en el acta.

Los respaldos administrados del proveedor no sustituyen una copia logica exportable bajo control del cliente.

6.1 Crear respaldo logico

Ejecute desde un equipo seguro con PostgreSQL Client instalado:

```
pg_dump --format=custom --no-owner --no-acl --file=tiendita_YYYYMMDD_HHMM.dump
"<DATABASE_URL>"
```

Despues:

1. Verifique que el archivo no este vacio.
2. Calcule y registre una suma SHA-256.
3. Cifre o almacene el archivo en un repositorio cifrado.
4. Registre fecha, entorno, tamano, responsable y resultado.
5. Aplique la retencion sin eliminar la ultima copia valida mensual.

6.2 Respaldo previo a despliegue

Siempre cree un respaldo antes de migraciones, cambios masivos de catalogo o correcciones directas de datos. Etiquete la copia con el commit y motivo del cambio.

7. Restauracion controlada

Restaurar reemplaza o combina datos segun el destino. Nunca practique sobre produccion. Primero use una base nueva y aislada.

1. Declare el incidente y detenga escrituras si existe riesgo de corrupcion.
2. Seleccione el respaldo segun fecha, integridad y objetivo de recuperacion.
3. Cree una base PostgreSQL vacia en un entorno aislado.
4. Restaure con `pg_restore --clean --if-exists --no-owner --no-acl --dbname="<TEST_DATABASE_URL>" archivo.dump`.
5. Ejecute las migraciones de la version de aplicacion que se utilizara.
6. Compruebe usuarios, productos, inventario, pedidos, caja y totales.
7. Documente cuantas transacciones quedarian fuera del respaldo.
8. Obtenga autorizacion del propietario antes de sustituir produccion.
9. Cambie la conexion, despliegue y ejecute pruebas de humo.
10. Conserve evidencia y realice un informe posterior al incidente.

7.1 Criterios de una restauracion aprobada

- La base abre sin errores de esquema.
- Las cuentas esperadas existen y los permisos funcionan.
- Los conteos de productos, pedidos y movimientos son plausibles.
- Una venta de prueba y un ajuste de inventario se completan.
- No se utiliza el modo live de pagos durante la prueba.
- La fecha restaurada y la perdida maxima de datos son conocidas.

8. Monitoreo e incidentes

Revise disponibilidad del frontend, salud del backend, errores HTTP, conexiones PostgreSQL, fallas de migracion, correo, imagenes y pagos. Nunca incluya contraseñas, tokens ni datos completos de clientes en logs o tickets.

Severidad	Ejemplo	Respuesta inicial sugerida
Critica	Sitio caido, cobros inconsistentes o perdida de datos.	Contener, avisar al propietario y preservar evidencia inmediatamente.
Alta	Caja o checkout bloqueado para varios usuarios.	Diagnosticar durante horario acordado y definir alternativa operativa.
Media	Un modulo secundario falla o existe degradacion.	Registrar, reproducir y programar correccion.
Baja	Consulta, texto o mejora cosmetica.	Incluir en backlog.

8.1 Rollback de aplicacion

Si el esquema sigue siendo compatible, vuelva a desplegar el commit anterior en Railway y verifique la API antes del frontend. Si una migracion cambio datos o estructura de forma incompatible, no revierta a ciegas: restaure una copia aislada, determine impacto y obtenga autorizacion.

9. Transferencia y salida del proveedor

- ☐ El cliente es propietario o administrador de GitHub, Railway, dominio y proveedores.
- ☐ Se rotaron secretos compartidos durante el desarrollo.
- ☐ El cliente recibio un respaldo reciente y comprobe su integridad.
- ☐ Existe una lista de renovaciones y fechas de pago.
- ☐ Los canales de soporte publicados son reales.
- ☐ Se documentaron servicios de terceros, costos y limites.
- ☐ Se retiro acceso de personas que ya no requieren administracion.

10. Registro de respaldos

Fecha/hora	Entorno	Archivo/ID	Tamano	SHA-256	Responsable	Restauracion probada
_____	_____	_____	_____	_____	_____	Si / No
_____	_____	_____	_____	_____	_____	Si / No
_____	_____	_____	_____	_____	_____	Si / No